Wheeler Memory Demo Script

Project Darman

March 2026

Wheeler Memory Demo Script

Part 1: Presentation Talking Points

Slide 1: Title Slide

“Wheeler Memory: A Memory System That Remembers Like You Do”

Talking Points (30 seconds): - Welcome the audience. This is a story about memory—human memory, artificial memory, and how they’re more similar than you’d think. - Wheeler Memory is a new kind of memory system for AI that actually forgets, admits uncertainty, and reconstructs memories differently depending on context. - Named after John Wheeler’s “It from Bit”—the idea that information emerges from dynamics, not static storage. - Today you’ll see how it works, why it matters, and what we’ve built.

Slide 2: The Hook

“Why do you remember your first kiss differently every time you think about it?”

Talking Points (45 seconds): - Context shapes memory. The same event, recalled at different times, in different moods, with different people—each recall is unique. - You’re not retrieving a fixed memory from storage. You’re **reconstructing** it, using context as a lens. - That’s a feature, not a bug. Your memory adapts to what matters right now. - This is the first time I really understood that I wasn’t remembering—I was reconstructing. And that changed how I think about AI.

Slide 3: The Problem With Today’s AI

Talking Points (1 minute): - ChatGPT can’t remember you between conversations. Start a new chat, it doesn’t know who you are. - More importantly: it never forgets. It never says “I’m not sure.” It confabulates confidently. It hallucinates. - A human says “I think that happened in 2015, but I’m not sure”—admitting uncertainty grounds the conversation in reality. - An LLM says “Yes, that was in 2015” with absolute confidence—even when it’s making it up. - We want AI that can **admit it doesn’t remember clearly**. That’s epistemic honesty.

Slide 4: What If AI Could Actually Remember?

Talking Points (45 seconds): - What if an AI system could forget things? Fade memories that aren't used? Admit uncertainty about old memories? - What if it could hedge: "I vaguely recall… it might have been…" instead of confident confabulation? - What if the same memory could be reconstructed differently depending on context—like your brain does? - That's the vision. That's Wheeler Memory.

Slide 5: Cellular Automata

Talking Points (1.5 minutes): - Most people know Conway's Game of Life—a grid of cells, simple rules, but complex emergent behavior. - Wheeler Memory uses the same idea, but instead of "alive/dead," we have three states: +1, 0, −1. - Three rules per cell: - If you're at a local peak (surrounded by lower values), push +1. - If you're at a local valley (surrounded by higher values), push −1. - If you're on a slope, flow uphill toward the peak. - You run these rules for ~40 to 100 ticks. Chaos settles. The grid converges to a stable pattern—an **attractor**. - That pattern **is** the memory. It's not stored as text or numbers. It's stored as the shape the automaton settles into. - This is inspired by real neuroscience: your brain stores memories as patterns of neural activity, not text files.

Slide 6: How Storing Works

Talking Points (1 minute): - You have a memory you want to store: "I learned about cellular automata in a coffee shop in 2024." - We hash that text (SHA-256), feed it as initial conditions to the CA, and let it evolve. - In 40–100 ticks (~3 milliseconds), chaos settles into a stable pattern. - That pattern is stored as a file on disk—just numbers, just the shape. - On a GPU, this is 10× faster. But even on CPU, it's fast enough for real-time use.

Slide 7: How Recalling Works

Talking Points (1.5 minutes): - You ask a question: "What did I learn about cellular automata?" - The system searches memory for the closest stored pattern. How? Pearson correlation—a statistical measure of similarity. - But here's the magic: instead of just returning the stored pattern, it **blends** it with your query context. - The blend is: 70% stored memory + 30% query context. We re-evolve that blend through the CA. - Out comes a reconstruction.

The same stored memory, colored by your current question. - If you asked “What coffee shops have I been to?” instead, the same stored memory would reconstruct differently. Different context, different recall.

Slide 8: Reconstruction

Talking Points (1 minute): - This is the core insight: **Same memory, infinite reconstructions.** - Your brain does this. Every time you recall a memory, you’re reconstructing it. A little changes each time. - That’s why eyewitness testimony is unreliable—not because people are lying, but because memory is reconstructive. - Wheeler Memory makes this explicit. It admits that the same stored pattern can be recalled in different ways. - That’s more honest than pretending each recall is the same.

Slide 9: Temperature – Forgetting

Talking Points (1.5 minutes): - Memories don’t live forever. They have a temperature. - New memories start **cool**. You’ve just heard something; you’re not confident about it yet. - Frequently used memories get **warm**. You’ve thought about them several times; you’re confident, but not certain. - Memories you use constantly become **hot**. You remember them clearly. - Age matters too. Every 7 days without recall, a memory loses temperature (half-life decay). It cools down. - Three tiers: - Hot (≥0.6): “I distinctly remember…” - Warm (≥0.3): “I believe… but I’m not entirely certain…” - Cold (<0.3): “I vaguely recall… it might have been…”

Slide 10: Epistemic Humility

Talking Points (1 minute): - Temperature directly affects the LLM’s language. The system mirrors memory freshness. - A hot memory gets phrased with confidence. A cold memory gets hedged. - This is revolutionary because current AI never hedges. It confabulates at the same confidence level regardless of how certain it should be. - Wheeler Memory lets you know: “This is a solid memory” vs. “This is a vague recollection.” - That’s epistemic humility. That’s honesty.

Slide 11: Why This Matters

Talking Points (1 minute): - Current LLMs confabulate confidently. They'll invent facts and state them as truth. - Wheeler Memory lets AI say "I'm not sure" and **mean it**. Confidence is grounded in memory freshness. - This is grounded in theory: Wheeler's information dynamics, cellular automata, and Loftus's research on reconstructive memory. - It's validated in practice: 10K+ test cases, 95%+ semantic coverage, fully tested.

Slide 12: Not Just Search

Talking Points (1 minute): - Most "memory" systems for AI are vector databases. You store embeddings, search by similarity, retrieve exact matches. - That's retrieval, not memory. It's a filing cabinet. - Wheeler Memory is reconstruction. Find the closest pattern, blend it with context, re-evolve it. - Meaning emerges from the dynamics, not from stored labels. - It's fundamentally different.

Slide 13: Two Modes: Exact + Fuzzy

Talking Points (1 minute): - Exact mode: Store "buy milk." Query "buy milk." Get the exact memory back. - Fuzzy mode: Store "grocery list" (as a memory about shopping). Query "what do I need to buy?" Get a context-colored reconstruction. - Same system, two philosophies. You can switch based on what you need.

Slide 14: The Philosophical Foundation

Talking Points (1 minute): - John Wheeler: "It from Bit." Information emerges from dynamics, not storage. - A memory isn't retrieved. It's **computed**. Reconstructed. Born anew each time you recall it. - Elizabeth Loftus: Human memory is reconstructive, not reproductive. We rebuild our memories each time. - That's the foundation. Everything else is commentary.

Slide 15: Design Philosophy

Talking Points (1 minute): - **Engine**: The cellular automaton (the mind). - **Voice**: The LLM wrapper (the speaker). - These are separate. Swap the LLM, the memory system stays the same. - Swap the CA rules, the LLM behavior changes accordingly. - Meaning comes from structure, not from labels or prompts.

Slide 16: Privacy & Ownership

Talking Points (45 seconds): - All local. No cloud APIs. No third-party servers. No subscription. - Your memories live on your machine. Your data stays private. - You own your mind. The automaton runs locally. No one else sees it.

Slide 17: Real Numbers

Talking Points (1.5 minutes): - 10K+ test cases across 6 domains: code, hardware, daily_tasks, science, meta, general. - 95%+ paraphrase coverage. The system handles semantic variations well. - ~3 milliseconds for convergence on CPU. GPU: 10× faster. - 19 modules, 167 tests, fully open-source. - Validated in practice. Not vaporware.

Slide 18: What's Running Today

Talking Points (1 minute): - Web UI dashboard. You can interact with it in real time. - CLI tools: `wheeler-store` (store a memory), `wheeler-recall` (recall), `wheeler-temps` (check temperatures), `wheeler-scrub` (delete old memories). - Darman chatbot agent. Ask it questions; watch it auto-recall memories. - Sleep consolidation. Spreading activation via offline consolidation. - Semantic embeddings. Optional; core recall works without ML. - GPU auto-fallback. Tries GPU; falls back to CPU gracefully. - Everything is local.

Slide 19: Why This Excites Us

Talking Points (1.5 minutes): - This is the first architecture to combine cellular automata with associative recall for AI. - It's rooted in theory: Wheeler's information dynamics, cellular automata, cognitive science. - It's validated in practice: thousands of tests, real-world performance metrics. - It gives AI a mind that remembers like you do: imperfectly, associatively, context-dependently. - It's open-source. It's local. It's yours.

Slide 20: Closing

Talking Points (45 seconds): - "The formula is the foundation. Everything else is commentary." - Wheeler Memory isn't a search engine. It's not a retrieval system. - It's a reconstruction engine. A mind. An automaton. A voice. - Darman remembers.

Part 2: Live Walkthrough Script

Prerequisites

Before starting the demo, ensure: - Web UI is running: `python -m wheeler_memory.web` (or `cd /home/tristan/wheeler\ memory && python -m wheeler_memory.web`) - Default data dir: `~/.wheeler_memory/` (or set `WHEELER_DATA_DIR` env var) - CLI tools are in PATH or run with `python -m wheeler_memory.scripts.wheeler_store`, etc. - Ollama is running (if demoing Darman agent): `ollama serve` - Test data is loaded (see troubleshooting)

Total Demo Time: ~8–10 minutes

Section 1: Web UI Dashboard (2–3 minutes)

Goal: Show the UI, store a memory, recall it, observe temperature decay.

Step 1.1: Open the Dashboard

- Open browser to `http://localhost:5000` (or configured port).
- Show the main dashboard: memory stats, recent memories, temperature gauge.
- **Talking Point**: “This is your memory system. In real time, you can see all your stored memories, their temperatures, their age.”

Timing: 20 seconds

Step 1.2: Store a Memory

- Navigate to “Store Memory” form.
- Input text: *“I learned that cellular automata can encode arbitrary information in stable patterns. This is inspired by work at the Santa Fe Institute.”*
- Optional: Set domain to “science” or “general”.
- Click “Store”.
- **Talking Point**: “We’re hashing this text, evolving it through the CA, and storing the stable attractor pattern. Behind the scenes, the automaton is converging. When it settles, we save the pattern.”

Timing: 45 seconds (including evolution time)

Step 1.3: Observe the Stored Memory

- Refresh the dashboard. The memory appears in “Recent Memories.”
- Show the attractor stats: convergence time, temperature (should be ~0.6–0.8 if just stored), age (0 days).
- **Talking Point**: “This memory is **hot**. It’s just stored. The temperature is high. If I ask about it now, the system will reconstruct it with high confidence.”

Timing: 30 seconds

Step 1.4: Recall the Memory

- Navigate to “Recall Memory” form.
- Query: *“What did I learn about cellular automata?”*
- Click “Recall”.
- Show the result: reconstructed memory blended with query context.
- **Talking Point**: “The system found the closest stored pattern, blended it with the query, and re-evolved it. Same memory, reconstructed through the lens of ‘cellular automata.’ If I queried ‘What did I learn about the Santa Fe Institute?’ the reconstruction would be different.”

Timing: 1 minute

Step 1.5: Check Temperature Over Time (Optional)

- If time permits, navigate to “Temperature View”.
- Show temperature gauge for the stored memory.
- **Talking Point**: “Every 7 days without recall, this memory loses temperature (half-life decay). If I never ask about cellular automata again, in a month this memory will be cold, and the system will say ‘I vaguely recall…’ instead of ‘I remember clearly.’”

Timing: 30 seconds (optional)

Section 2: CLI Tools (3–4 minutes)

Goal: Show the command-line interface for store, recall, temperatures, and scrub.

Step 2.1: Wheeler Store (CLI)

- Open terminal.

- Run:

```
python -m wheeler_memory.scripts.wheeler_store \
  "I went hiking in the mountains last weekend. The trail had
       wildflowers." \
  --domain daily_tasks
```

- Show output: hash, convergence stats, temperature.
- **Talking Point**: “That’s storing a memory via CLI. We get back the hash (the unique ID), convergence time, and temperature. This memory is hot.”

Timing: 45 seconds

Step 2.2: Wheeler Recall (CLI)

- Run:

```
python -m wheeler_memory.scripts.wheeler_recall \
  "Where did I go last weekend?" \
  --top_k 3
```

- Show results: top 3 closest memories, their temperatures, reconstructed blends.
- **Talking Point**: “Three memories came back. The hiking memory is the closest match. See the temperature? It’s hot, so the recall is confident. If we queried ‘What wildflowers are in the mountains?’ the reconstruction would emphasize different aspects.”

Timing: 1 minute

Step 2.3: Wheeler Temps (CLI)

- Run:

```
python -m wheeler_memory.scripts.wheeler_temps --top_n 5 --sort hot
```

- Show the 5 hottest memories, their temperatures, ages.
- **Talking Point**: “This shows the heatmap of your memories. Hot ones are recent or frequently recalled. Cold ones are old or forgotten. The system can use this to prioritize what to consolidate during sleep.”

Timing: 45 seconds

Step 2.4: Wheeler Scrub (CLI, Optional)

- Show the scrub command (don't execute unless you want to delete demos):

```
python -m wheeler_memory.scripts.wheeler_scrub --age 30 --confirm
```

- **Talking Point**: "This deletes memories older than 30 days. Privacy and cleanup in one command. Darman never accumulates stale memories."

Timing: 20 seconds

Section 3: Darman Agent (3–4 minutes)

Goal: Show the interactive agent loop. Ask it questions, watch it auto-recall, observe epistemic hedging.

Step 3.1: Start Darman

- Run:

```
python -m wheeler_memory.agent --interactive --model ollama/mistral
```

- Show the startup message: agent is ready, memory system is online, Ollama is connected.

- **Talking Point**: "This is Darman—the voice for Wheeler Memory. It's running locally. It has memory. Ask it something."

Timing: 30 seconds

Step 3.2: Ask a Hot Memory Question

- Ask: *"What did I learn about cellular automata?"*
- Darman auto-recalls the memory (watch the system auto-search and reconstruct).
- Show the response, which references the memory with confidence: *"You learned that cellular automata can encode arbitrary information in stable patterns, inspired by the Santa Fe Institute…"*

- **Talking Point**: “The memory is hot, so Darman speaks with confidence. Notice the phrasing: ‘You learned…’ It’s not hedging. The temperature is high.”

Timing: 1 minute

Step 3.3: Ask a Warm/Cold Memory Question

- Ask: *“Tell me again about that hike—I forget the details.”*
- Darman auto-recalls the hiking memory (if temperature has cooled, show it).
- Show the response with hedging: *“I believe you went hiking… the trail had wildflowers, though I’m less certain about other details…”*
- **Talking Point**: “Temperature affects the phrasing. If this memory is a few days old, the system admits uncertainty. Confidence is honest.”

Timing: 1 minute

Step 3.4: Ask About Something Not in Memory

- Ask: *“Did I ever attend a conference?”*
- Darman searches memory, finds nothing close, responds: *“I don’t recall you attending a conference. I have no memory of that.”*
- **Talking Point**: “The agent doesn’t confabulate. It admits what it doesn’t remember. That’s epistemic integrity.”

Timing: 1 minute

Step 3.5: New Memory During Chat

- Ask: *“Let me tell you something new: I’m learning about dissipative structures and self-organization.”*
- Darman prompts you to store this, or auto-stores it (depending on configuration).
- Show the memory stored in real time.
- **Talking Point**: “Darman is learning. New memories are stored hot. Old memories fade. The system evolves.”

Timing: 1 minute

Exit Darman

- Type `exit` or `quit` to exit the agent loop.

Section 4: Under the Hood (2–3 minutes)

Goal: Show the CA evolution visually and explain convergence.

Step 4.1: Show Evolution GIF

- Open `/home/tristan/wheeler memory/docs/assets/diagrams/evolution.gif` in an image viewer or browser.
- **Talking Point**: “This is a cellular automaton evolving over time. You see chaos at the start (random initial state). As the rules apply, order emerges. Peaks push outward, valleys pull inward, slopes flow uphill. In ~100 ticks, it converges to a stable pattern. That pattern is the memory.”

Timing: 1 minute

Step 4.2: Show Reconstruction Demo

- Open `/home/tristan/wheeler memory/docs/assets/reports/reconstruction_demo.png` in an image viewer.
- **Talking Point**: “This shows three reconstructions of the same stored pattern, blended with different query contexts (α=0.3). Same memory, three different reconstructions. The left might be the stored pattern. The middle blends with one query. The right blends with another. Context colors recall.”

Timing: 45 seconds

Step 4.3: Explain Convergence States

- Open a terminal and run a quick analysis (if available):

```
python -m wheeler_memory.scripts.wheeler_stats
```

- Show convergence state distribution: CONVERGED, OSCILLATING, CHAOTIC.
- **Talking Point**: “Most memories converge to stable patterns (CONVERGED). Some get stuck oscillating between states (OSCILLATING). A few never settle (CHAOTIC). The system tracks all three. Convergence quality matters for recall fidelity.”

Timing: 45 seconds

Troubleshooting & Contingencies

Web UI Not Loading

- Check if service is running: `curl http://localhost:5000`
- Restart: `pkill -f "python -m wheeler_memory.web"`, then restart.
- If port 5000 is in use, set `WHEELER_WEB_PORT=5001` and retry.

CLI Tools Not Found

- Ensure package is installed: `pip install -e /path/to/wheeler_memory`
- Alternatively, run with full module path: `python -m wheeler_memory.scripts.wheeler_store ...`

Ollama Not Connected

- Ensure Ollama is running: `ollama serve` in a separate terminal.
- Check Ollama models: `ollama list`. If no models, pull one: `ollama pull mistral`
- If still failing, demo Darman with `--mock-ollama` flag for demo responses (if available).

No Data in Memories

- Test data may need to be loaded. Run:

```
python -m wheeler_memory.scripts.wheeler_load_test_data
```

- Or manually store a few memories first (Step 2.1).

GPU Not Detected

- If running on machine with GPU, check: `python -c "import torch; print(torch.cuda.is_available())"`
- The system auto-falls back to CPU if GPU unavailable. No manual intervention needed.
- For demo purposes, CPU is fast enough (~3ms).

Image Assets Not Found

- Verify paths:

```
ls -la /home/tristan/wheeler\ memory/docs/assets/diagrams/evolution.gif
ls -la /home/tristan/wheeler\ memory/docs/assets/reports/
       reconstruction_demo.png
```

- If missing, note in demo that these are generated by test runs or can be skipped.

Demo Script Summary

Section	Duration	Key Points
Web UI	2–3 min	Dashboard, store, recall, temperature
CLI Tools	3–4 min	wheeler-store, -recall, -temps, -scrub
Darman Agent	3–4 min	Hot memory (confident), warm/cold (hedged), confabulation prevention
Under the Hood	2–3 min	Evolution GIF, reconstruction demo, convergence states
Total	**~8–10 min**	Live, interactive, memorable

Notes for Presenter

1. **Pacing**: Move quickly through each section. If a question derails you, note it and move on. You can do deep dives afterward.
2. **Confidence**: You're showing something real. Use that. This isn't a mockup or a video. It's live code running.
3. **Human Touch**: Emphasize the reconstruction insight. "Same memory, different recalls" is the core idea. If people only remember that, you've succeeded.
4. **Local Privacy**: Reinforce that nothing leaves the machine. No cloud, no logs, no third parties. That resonates.
5. **Temperature Metaphor**: Hot/warm/cold is intuitive. Use it. "This memory is cold because I haven't thought about it in a while. If I recalled it now, I'd hedge. Like you would."

6. **Questions**: If someone asks about comparison to vector DBs or traditional memory systems, say: “Those retrieve. We reconstruct. Retrieval is fast but static. Reconstruction is slower but adaptive, like human memory.”